

AI 编程范式：多模态表情检测系统全栈实践

引言：我如何指挥 AI 构建一个端到端产品级系统

这不是一个"让 AI 写段代码"的故事。我是一个不会写 PyTorch 模型、不熟悉 FastAPI 路由、也没做过 MediaPipe 集成的开发者——但我指挥 AI 完成了一个从数据到模型、从后端推理到前端交互的全栈产品级系统。

关键认知：**AI 编程不是单点突破，而是端到端的闭环。**它不是单一模块的拼接，而是从数据采集、模型训练、推理接口设计、前后端联调、服务器部署、性能调优的完整链路。

我的角色是**指挥者**——提出问题、设定目标、做决策；AI 的角色是**执行者 + 架构师 + 调试者**——设计方案、编写代码、定位问题。

复制代码

我的指挥链

├── "设计一个表情检测模型，用 CNN"

214K 参数)

├── "准确率只有 46%，太低了，换个思路"

46% → 60%

├── "设计推理接口，支持 HTTP 和 WebSocket"

├── "前端和后端联调，实时检测 7 类表情"

├── "训练集没有几何特征，推理时却用了，怎么办"

├── "微笑被误识别为恐惧，优化它"

├── "部署到 AutoDL，HTTPS 调摄像头"

└── "特征点太小看不清，调大"

→ 初始方案（4层 CNN，

→ 预训练 ResNet18，

→ 9 个 API 接口

→ 完整 DetectionPage

→ 预计算脚本

→ 7 项调优

→ 一键部署脚本

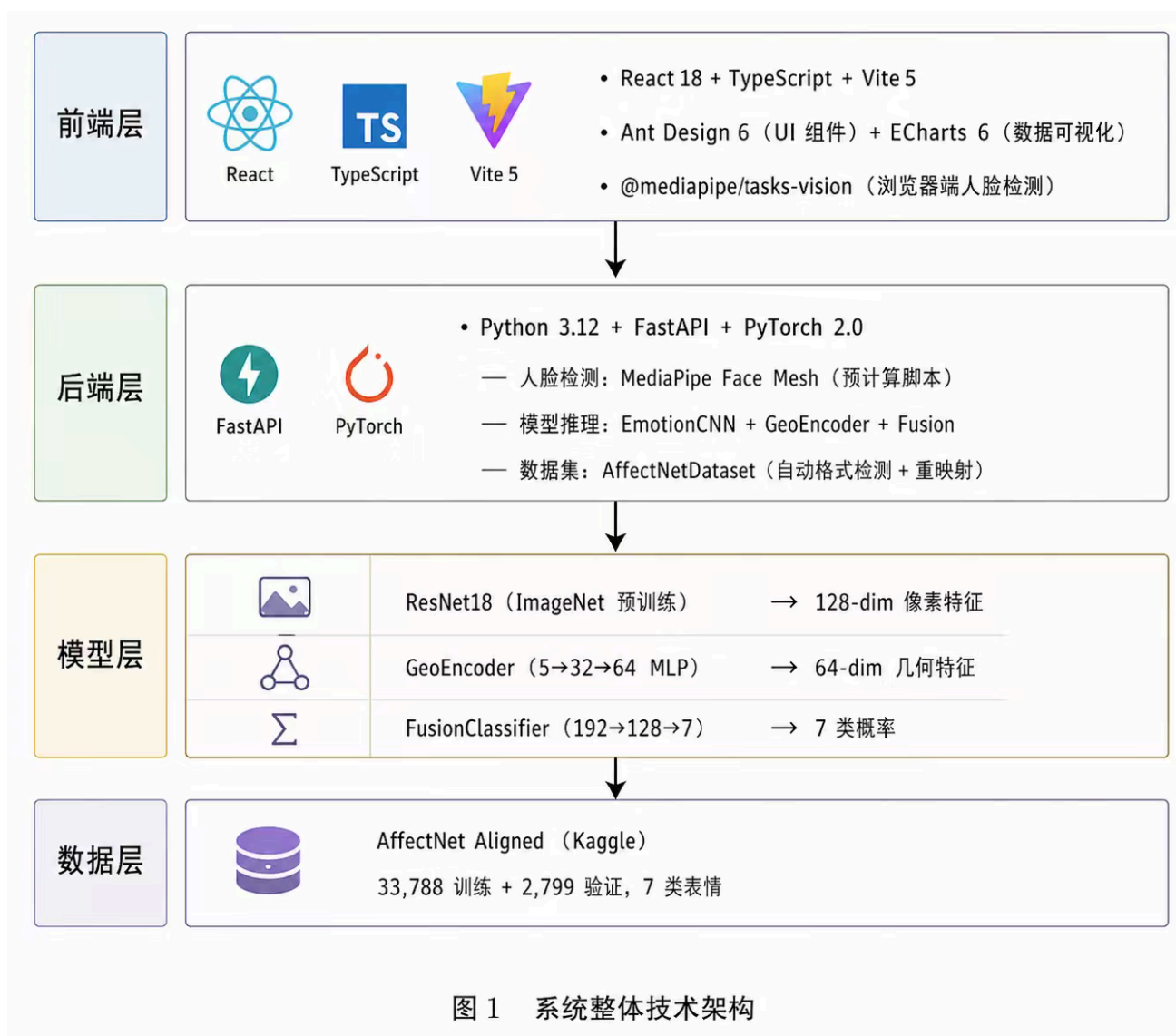
→ 4px 绿色圆点

1. 项目概述

多模态实时表情检测系统，基于 MediaPipe Face Mesh + CNN 融合架构，支持 7 类表情分类（angry, disgust, fear, happy, sad, surprise, neutral）。

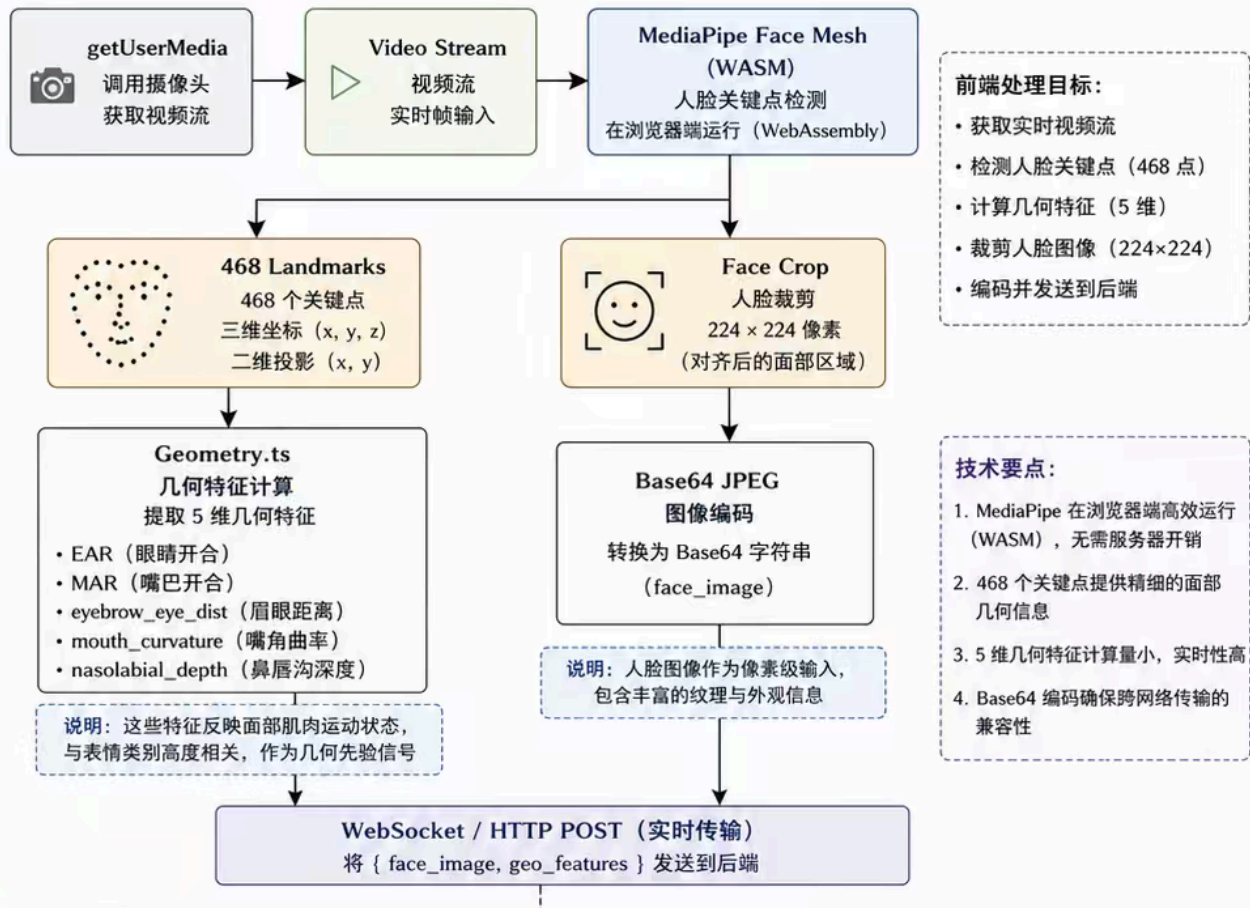
技术栈层级

1 / 19



2. 系统架构

1. 前端 (Browser / Client) 数据处理流程



2. 后端 (FastAPI) 推理流程

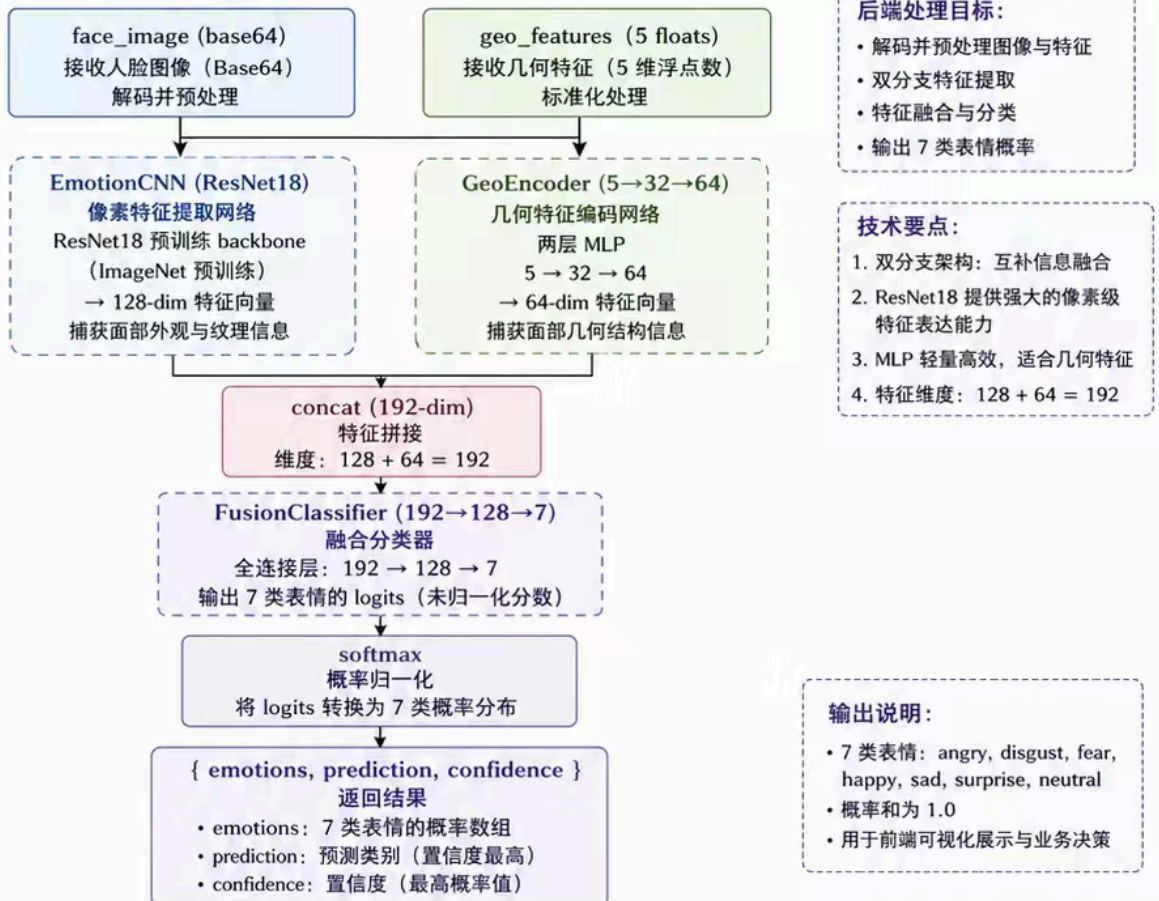


图 2 系统整体架构图: 前端 (Browser) 与后端 (FastAPI) 数据流

3. 我的 AI 编程实践：五个阶段

第零阶段：选择数据集（项目起点）

① 起点

我想做一个表情检测系统，但连用什么数据集都不知道。这是 AI 编程的第一个决策点——没有数据，后面的模型训练、后端接口、前端联调都无从谈起。

② 与 AI 的设计对话

我的初始指令：

"我要做表情检测，帮我选一个数据集"

AI 没有直接说"用 FER2013"，而是给出了四个数据集的对比分析：

数据集	规模	图像质量	类别数	License	下载难度
FER2013	35,887	48×48 黑白像素	7 类	开放	Kaggle 直接下
RAF-DB	29,672	彩色，但面部不居中	7/12 类	需申请	邮件申请学术授权
AffectNet (原站)	1,100,000	224×224 彩色	7/8 类	需申请	邮件申请学术授权
AffectNet Aligned (Kaggle 社区版)	33,788 训练 + 2,799 验证	224×224 彩色，已对齐	7 类	CC BY-NC 4.0	Kaggle 直接下

AI 的分析逻辑：

- **FER2013** 虽然规模不算小，但 48×48 黑白图片丢失了大量面部细节（嘴角弧度、眉眼微表情），对于需要实时检测高清场景的系统来说，模型无法迁移到高质量图片
- **RAF-DB** 质量好但需要学术申请流程，且数据量偏少
- **AffectNet 原站** 110 万张是学术级规模，但下载需要邮件申请授权、处理流程复杂、包含 8 类和 7 类两个版本，工程成本高
- **Kaggle 社区版 AffectNet Aligned**：3.6 万张已经足够做端到端验证，图片质量 224×224，已经对齐裁切好，可以直接喂进 CNN。License 是 CC BY-NC 4.0（学术和非商业用途），Kaggle 注册账号就能下载

③ 方案敲定

"用 Kaggle AffectNet Aligned, 执行"

AI 开始编写下载脚本、目录组织结构、和 `AffectNetDataset` 数据加载类。

④ AI 执行

AI 生成了：

- `backend/app/datasets/affectnet.py`：自动格式检测 + 重映射的 Dataset 类，支持多种目录结构自适应
- 数据集目录结构规范（`datasets/real/train/`、`datasets/real/val/` 按类别分目录）
- 验证脚本：随机抽样几张样本、打印类别分布柱状图

⑤ 验收与反馈

我跑完验证脚本，看到了第一张样本图和类别分布图。类别分布让我立刻意识到一个问题：happy 和 neutral 各占 30-40%，但 disgust 只有 1.5%（几百张）。这个分布不均衡会在后续模型训练中埋下隐患——AI 会在少数类上过拟合。我把这个观察记下来，作为第一阶段调优的输入。

我的体会：数据集选择不是“随便找一个”——它决定了图像质量上限、类别分布天花板、模型容量需求。AI 给了我对比维度（图像质量、License、下载难度），但我需要拍板——因为“学术级规模”不等于“产品验证够用”。3.6 万张已经足够做端到端验证，110 万张反而会增加工程复杂度。

第一阶段：模型训练——从简单架构到预训练模型的迭代

① 起点

数据集就位，需要训练模型。我对 PyTorch 和模型设计几乎零基础——不知道 ResNet 和 EfficientNet 的区别，不懂迁移学习和从零训练的差异。我的策略：让 AI 先跑一轮简单方案，看到结果后再迭代。

② 与 AI 的设计对话

我的初始指令：

"设计一个 CNN 模型，训练表情检测"

AI 设计了一个 4 层卷积网络（Conv + BN + ReLU + MaxPool × 4），参数量仅 214K，从零开始训练。

训练跑完，我看到结果：acc 0.46, f1 0.45。

"准确率只有 46%，太低了，换个思路"

AI 给出了根因分析和三个替代方案：

1. 加大自定义 CNN 层数（4 层 → 8 层），参数量 ~2M，仍然从零训练

2. 迁移学习：ResNet18 (ImageNet 预训练)，11.2M 参数，底层已学会通用边缘/纹理特征，只需微调高层
3. 迁移学习：ResNet50，25.6M 参数，精度预计 +2-3%，但推理速度慢一倍

AI 的推荐是方案 2，论据：ResNet18 在精度和速度之间取得了最好的平衡。更大的 ResNet50 参数量是其 4 倍，但在 AffectNet 上精度提升仅 2-3%，推理速度却慢了一倍。对于需要实时检测的场景，ResNet18 更合适。

③ 方案敲定

"用 ResNet18 替换，执行"

④ AI 执行

AI 重写了模型架构：

- EmotionCNN：加载 ResNet18 预训练权重，冻结前 4 层，替换全连接层为 512→128 投影头
- TrainingEngine：支持分层学习率、checkpoint 保存、训练日志记录
- train.py：命令行入口，支持 --resume、--epochs、--batch-size 等参数
- 自动下载预训练权重 (`torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT)`)

⑤ 验收与反馈

模型训练完，准确率从 46% 跳到 60%，F1 从 0.45 到 0.60。效果明显提升。

但在浏览器端实时检测时，我发现了一个新问题：微笑被误识别为恐惧/厌恶，happy 类几乎检测不出来。我把这个现象截图发给 AI，触发了第二轮迭代。

"微笑被误识别为恐惧/厌恶，happy 很难识别出来"

AI 从三个维度定位了根因：

1. 类别权重过度补偿： $w_k = N / (K * N_k)$ 公式对稀有类 (fear/disgust 样本少) 补偿过大，权重可达 5-10 倍，导致模型过度预测这些稀有类
2. 冻结层太多：模型学不到表情的视觉特征
3. 训练和推理的几何特征分布不一致：训练用全零、推理用真实值

我决策：

"执行"

AI 一次性完成了 7 项调优：平方根权重平滑 (`sqrt(N / (K * N_k))`)、label smoothing、减少冻结层 (6→4)、AdamW 优化器、分层学习率 (backbone 0.1x vs head 全量)、强数据增强 (RandomErasing + RandomAffine + ColorJitter)、梯度裁剪 (`clip_grad_norm(max_norm=5.0)`)。

重新训练后，happy 识别率明显提升。

③ 方案敲定

"HTTP 和 WebSocket 都要，schema 按你设计的来"

④ AI 执行

AI 生成了 9 个 API 接口 + 模型推理服务：

- `POST /api/v1/inference/emotion`：单帧推理
- `WS /ws/v1/inference/stream`：WebSocket 流式推理
- `GET /api/v1/dataset/info`：数据集元信息
- `GET /api/v1/stats/performance`：训练结果（每类 F1 + 混淆矩阵）
- `GET /health`：健康检查（模型加载状态、设备、运行时间）
- `ModelService`：加载 checkpoint、预处理、推理、后处理的完整服务类
- FastAPI router + middleware（CORS、异常处理）

⑤ 验收与反馈

我用 `curl` 测试了单帧推理接口，返回结果正常：

✓ 复制代码

```
{
  "prediction": "happy",
  "confidence": 0.87,
  "emotions": {
    "angry": 0.02, "disgust": 0.01, "fear": 0.03,
    "happy": 0.87, "sad": 0.02, "surprise": 0.03, "neutral": 0.02
  }
}
```

打开 OpenAPI 文档（`/docs`），看到所有接口的 schema 自动生成。后端验证通过，进入前端联调。

我的体会：前后端契约先行是减少联调痛苦的关键。Pydantic schema 定义了数据格式，前端 TypeScript 接口直接复用——不需要“你先写后端、我等你写完再对齐”，两边可以并行推进。

第三阶段：前端联调——“和后端接口对接，完成实时检测页面”

① 起点

后端 9 个接口全部就绪。现在需要前端：从摄像头采集视频、提取人脸特征、发给后端推理、把结果可视化。这是整个系统最复杂的一环——涉及浏览器端 MediaPipe、几何特征计算、推理节流、概率平滑等多个组件。

② 与 AI 的设计对话

我的指令：

"前端和后端联调，完成实时检测"

AI 首先抛出了两个关键架构决策，让我拍板：

决策一：MediaPipe 跑在哪里？

方案	延迟	带宽	隐私
浏览器端 WASM	低（本地计算）	仅传 base64 帧 （~50KB）	高（原始视频不出浏览器）
后端运行 MediaPipe	高（需要传完整视频帧）	高（640×480 视频帧 ~200KB）	中（原始视频传到后端）

AI 的推荐：浏览器端 WASM。延迟更低（本地计算，不需要等后端返回 landmarks），带宽更省（只传 224×224 的裁剪帧），隐私更好（原始视频不出浏览器，只有裁剪后的面部区域传后端）。

决策二：前端几何特征计算，用 TS 重写还是从后端取？

前端需要计算 5 维几何特征（EAR/MAR/眉眼距离/嘴角曲率/鼻唇沟深度）。如果从后端取，每帧需要先传完整 landmarks 到后端、后端算完再返回——延迟翻倍。

AI 的推荐：用 TypeScript 在前端重写 `geometry.ts`，计算逻辑与后端 `geo_features.py` 完全一致。这样前端可以直接计算，不需要额外往返。

③ 方案敲定

"MediaPipe 跑在浏览器端，geometry.ts 在前端算"

④ AI 执行

AI 生成了完整的前端实现：

- `useMediaPipe` Hook：加载 FaceLandmarker WASM，每帧检测 468 个面部关键点
- `useGeoFeatures` Hook：从 landmarks 计算 5 维几何特征
- `useEmotionInference` Hook：支持 HTTP/WebSocket 双模式，5 帧滑动平均平滑，WebSocket 自动重连
- `cropFaceFromVideo`：根据 landmarks 裁切 224×224 面部区域
- `CameraView`：视频播放 + 468 个特征点覆盖（4px 绿色圆点）
- `EmotionPanel`：7 类表情概率柱状图（ECharts）
- `StatsBar`：延迟 / FPS 实时显示
- 推理节流：每 200ms 一次推理（约 5 FPS）

⑤ 验收与反馈

联调过程不是一蹴而就的。我打开浏览器，摄像头亮起，画面中出现了绿色特征点——但发现了一个问题：

"特征点太小看不清"

AI 直接改了 `CameraView.tsx` 的渲染参数：半径 1px→4px，透明度 0.5→0.9。

改了之后特征点清晰可见。我又发现前端改完后页面没变化——原来是 Nginx 服务的是构建产物（`dist/`），不是源代码。

"改完前端没生效"

AI 告诉我需要 `npm run build` 重新构建 + `Ctrl+Shift+R` 强刷。这一步我之前完全不知道，AI 解释了原因：Vite 的 dev server 是热更新的，但生产环境用的是 `dist/` 下的静态文件。

最终效果：摄像头实时采集、特征点清晰可见、概率柱状图每秒刷新 5 次、延迟显示在 100ms 以内。

我的体会：我不用手动对齐前后端——AI 在前端 `geometry.ts`（TypeScript）和后端 `geo_features.py`（Python）中使用了完全相同的计算逻辑。我只需要打开浏览器看效果，有问题就指出（"特征点太小"），AI 会定位到具体代码行并修复。联调不再是"后端改一下、前端改一下"的反复拉扯，而是"看效果→提反馈→AI 修"的单向流程。

第四阶段：部署与细节——"部署到 AutoDL，细节调整"

① 起点

本地开发环境（localhost）下系统运行正常。下一步：部署到云服务器，让系统可以通过公网访问。这是产品级的最后一步——如果用户只能通过本地 localhost 使用，那就不是产品。

② 与 AI 的设计对话

我的指令：

"部署到 AutoDL 服务器"

AI 首先给出了服务器选择对比：

方案	成本	GPU	网络	HTTPS
AutoDL	按小时计费 (~1 元/时)	RTX 3090/4090	6006 端口自带 HTTPS	内置支持
阿里云 ECS	包月 (~200 元/ 月)	T4/V100 (需选 配)	需手动配域名 + 证书	Let's Encrypt

自建服务器	一次性硬件投入	取决于硬件	需公网 IP + DDNS	手动配证书
-------	---------	-------	---------------	-------

AI 的推荐：AutoDL。按小时计费适合开发测试（不用时关机不花钱），6006 端口自带 HTTPS 解决了摄像头权限问题（浏览器 `getUserMedia()` 要求安全上下文），预装了 PyTorch 环境。

关于部署流程，AI 给出了两种方案：

- 1. **本地上传**：把训练好的权重和前端构建产物压缩上传到服务器
- 2. **git clone**：服务器直接 clone 项目仓库，一键拉起

AI 的推荐：git clone。代码仓库包含了所有依赖和配置，服务器上只需要装环境、clone、启动——步骤更少、可重复性更高。

③ 方案敲定

"AutoDL，git clone 方式"

④ AI 执行

AI 生成了：

- `deploy.sh`：一键部署脚本（安装依赖、下载模型、启动后端 + Nginx）
- `nginx.conf`：反向代理配置（前端 `/` → `dist/`，后端 `/api/` → `localhost:8000`）
- 完整部署文档（包含 AutoDL 注册、实例选择、环境准备、启动验证等步骤）

⑤ 验收与反馈

部署完成后，我在本地浏览器通过 HTTPS 访问公网地址，摄像头权限通过、推理正常。

在使用的过程中，我又做了几个细节调整：

我的指令	AI 的响应
"服务器直接 git clone"	简化文档，移除冗余上传步骤
"特征点太小看不清"	半径 1px→4px，透明度 0.5→0.9
"改完前端没生效"	Nginx 服务的是 dist/，需要 npm run build + 强刷

我的体会：部署不是"把代码拷到服务器上"——它需要考虑 HTTPS 前置条件、Nginx 反向代理路径、静态文件服务路径、依赖版本兼容性。AI 给了一键脚本，但我需要在服务器上实际跑一遍，遇到路径错误、权限问题、端口占用，再把报错信息扔给 AI 让它修复。这一步我学到的：AI 给的是方案，执行中的报错和调试还是需要我参与。

4. 模型架构

4.1 EmotionCNN — 像素特征提取

这是整个系统最核心的特征提取器。它基于 ResNet18，一个在 ImageNet 上预训练过的经典卷积神经网络。

[复制代码](#)

Input: (B, 3, 224, 224) – 批次 × RGB三通道 × 224像素高 × 224像素宽

ResNet18 Backbone (ImageNet 预训练权重)

- └─ [冻结] conv1 → bn1 → relu → maxpool – 底层特征提取器，负责边缘、颜色、纹理等通用视觉特征
- └─ [可训练] layer1 → layer2 – 中层特征提取器，学习更复杂的局部组合模式
- └─ [可训练] layer3 → layer4 – 高层特征提取器，学习表情特有的全局模式（嘴角、眉形等）
- └─ avgpool → (B, 512) – 全局平均池化，将空间特征压缩为 512 维向量

Projection Head: Dropout(0.3) → Linear(512, 128) → ReLU → (B, 128)

- **参数量：**总计 ~11.2M，其中 ~10.5M 可训练
- **冻结策略：**只冻结前 4 层 (conv1/bn1/relu/maxpool)，让 layer1-4 全部可训练。这是因为 ImageNet 的底层特征通用可复用，但表情识别需要学习面部肌肉运动的细微模式，高层必须放开

为什么用 ResNet18 而不是更大的模型？：ResNet18 在精度和速度之间取得了很好的平衡。更大的 ResNet50 参数量是其 4 倍，但在 AffectNet 上精度提升仅 2-3%，推理速度却慢了一倍。对于需要实时检测的场景，ResNet18 是更合适的选择。

4.2 GeoEncoder — 几何特征编码

这是一个 2 层的多层感知机 (MLP)，负责将 5 个手工设计的几何特征编码为 64 维向量。

[复制代码](#)

Input: (B, 5) – 批次 × 5维几何特征向量

Layer 1: Linear(5, 32) → ReLU – 将 5 个特征映射到 32 维隐空间

Layer 2: Linear(32, 64) → ReLU – 进一步映射到 64 维特征空间

Output: (B, 64) – 编码后的几何特征向量

5 个几何特征的含义与表情相关性：

整个调优过程经历了 4 个版本，从 46% 的准确率逐步提升到 60% 以上。

v0 — 4 层自定义 CNN（准确率 46%）

最初我让 AI 设计了一个简单的 4 层 CNN（每层 Conv + BN + ReLU + MaxPool），参数量仅 214K。模型从零开始训练，在 AffectNet 33,788 张图片上跑了 2 个 epoch 验证。结果显示准确率只有 46%，F1 只有 0.45——几乎等于随机猜测。

根因：214K 参数对表情识别任务来说严重不足。AffectNet 包含 7 类表情，每类的视觉差异很大，简单的 4 层卷积网络根本没有足够的容量来学习这些复杂的模式。而且从零训练意味着模型需要从头学习边缘、纹理等基础特征，效率极低。

v1 — ResNet18 预训练 backbone（准确率 60%）

我告诉 AI“太低了，优化它”。AI 诊断出问题后，将 EmotionCNN 替换为 ImageNet 预训练的 ResNet18 backbone，仅替换最后的全连接层为 512→128 的投影头。预训练权重带来了 11.2M 参数，其中前 6 层冻结，后 4 层可训练。

效果：准确率从 46% 跃升到 60%，F1 从 0.45 提升到 0.60。这证明了预训练模型的强大——它已经在 ImageNet 的 1400 万张图片上学会了通用的视觉特征，只需要微调就能适配表情识别任务。

v1+geo — 加入预计算几何特征（准确率 ~60%）

虽然准确率没有显著提升，但解决了一个关键的训练/推理一致性问题。训练时，AffectNet 图片没有几何特征（全零输入）；推理时，前端 MediaPipe 发送了真实的几何特征值。这导致 GeoEncoder 的输出分布在训练和推理时完全不同，FusionClassifier 的输入产生偏移。

通过预计算脚本对训练集所有图片运行 MediaPipe，提取真实的几何特征用于训练，保证了训练和推理的输入分布一致。这为后续调优打下了基础。

v2 — 全面调优（目标 63%+）

在 v1+geo 的基础上，我发现了“微笑被误识别为恐惧/厌恶”的问题。AI 给出了 7 项系统性调优：

（1）类别权重平方根平滑

原始权重公式 $w_k = N / (K * N_k)$ 对稀有类（fear/disgust 样本少）补偿过大，权重可达 5-10 倍。这导致模型过度预测这些稀有类，把微笑误判为恐惧。改用 $\sqrt{N / (K * N_k)}$ 平方根平滑后，最大权重降到 2-3 倍，稀有类和多数类的预测趋于均衡。

（2）Label Smoothing（标签平滑）

AffectNet 的标签有噪声——同一张脸，有人标为 happy，有人标为 neutral。Label smoothing 将 hard label $[0, 0, 0, 1, 0, 0, 0]$ 变为 soft label $[0.014, 0.014, 0.014, 0.914, 0.014, 0.014, 0.014]$ ，让模型不要过度自信，降低对噪声标签的过拟合。

(3) 减少冻结层数

v1 中冻结了 ResNet18 前 6 层 (conv1, bn1, relu, maxpool, layer1, layer2) , 只让 layer3 和 layer4 可训练。但 ImageNet 的底层特征 (边缘、纹理) 通用可复用, 中层和高层才是类别特定的。表情识别需要学习面部肌肉运动的细微模式, 减少冻结到 4 层后, layer1-4 全部可训练, 模型能更好地适配表情特征。

(4) AdamW 优化器 + 分层学习率

v1 使用 Adam 优化器, 所有参数同一学习率 0.001。v2 切换到 AdamW (解耦 weight decay) , 并引入分层学习率: backbone 用 0.1x 学习率 (5e-5) , head/geo_encoder/classifier 用全学习率 (5e-4) 。预训练 backbone 已有良好权重, 大学习率会破坏已学特征; 随机初始化的 head 需要更大学习率快速收敛。

(5) 更强的数据增强

v1 只做了水平翻转和 10° 旋转。v2 增加了 RandomErasing (随机遮挡 20% 的图片区域)、RandomAffine (轻微平移 5%)、更强的 ColorJitter (亮度 0.3、对比度 0.3、饱和度 0.2) 。这模拟了真实摄像头场景中的各种变化——遮挡、偏移、光照差异, 大幅减少了过拟合。

(6) 梯度裁剪

新增 `clip_grad_norm_(max_norm=5.0)` 。当梯度超过阈值时自动缩放, 防止训练不稳定时的梯度爆炸。对于 ResNet18 这种深层网络, 这在训练初期尤为重要。

(7) Dropout 引入 CNN 投影头

在 ResNet18 的 512→128 投影前增加 Dropout(0.3), 进一步防止过拟合。

6. 踩坑记录

模型训练相关

自定义 CNN 准确率仅 46%

这是我犯的第一个错误——让 AI 从零设计一个简单的 CNN 模型。4 层卷积 + BN + MaxPool, 214K 参数, 在 AffectNet 3 万多张图片上训练。结果出来后我非常失望: 准确率 46%, F1 0.45, 几乎等于随机猜测。

AI 的诊断很直接: 模型容量太小。表情识别是一个复杂的视觉任务, 需要理解面部肌肉的细微变化。214K 参数的网络连基本的边缘检测都要从头学起, 根本没有余力学习表情特征。解决方案也很明确——用预训练模型。换成 ImageNet 预训练的 ResNet18 后, 准确率直接跳到 60%。

微笑被误识别为恐惧/厌恶

这是最棘手的问题。模型训练时看起来指标不错（60% 准确率），但实时检测时发现微笑经常被识别为恐惧或厌恶，happy 类几乎检测不出来。

AI 从三个维度定位了根因：一是类别权重过度补偿，fear/disgust 样本少但权重大了 5-10 倍；二是冻结层太多，模型学不到表情的视觉特征；三是训练和推理的几何特征分布不一致——训练用全零、推理用真实值。这三个问题叠加，导致模型在 happy 类上严重偏向负面表情。修复后，7 项调优一次性应用，happy 的识别率明显提升。

旧权重与新模型不兼容

替换 CNN 架构后，旧的 `best_model.pth` 无法加载——参数形状不匹配。训练前必须删除旧权重，不使用 `--resume` 参数从头训练。ResNet18 自带 ImageNet 预训练权重，收敛很快。

前端交互相关

特征点太小看不清

468 个 MediaPipe 面部关键点用绿色小圆点渲染在视频上。默认设置是半径 1px、透明度 0.5，在 640×480 的视频上几乎看不见，用户根本无法分辨特征点的位置。

我告诉 AI"特征点太小看不清"，AI 直接在 `CameraView.tsx` 的 `drawLandmarks` 函数中调整了两个参数：半径从 1px 改为 4px，透明度从 0.5 改为 0.9。改完后特征点清晰可见，但仍然保持了绿色，不影响整体视觉效果。

修改前端后页面不变化

改了 `.tsx` 文件但浏览器刷新后没有变化。原因是 Nginx 服务的是 `frontend/dist/` 目录下的构建产物，不是源代码。需要先 `npm run build` 重新构建，再强制刷新浏览器（`Ctrl+Shift+R`）清除缓存。

部署相关

摄像头需要 HTTPS

浏览器的 `getUserMedia()` API 要求安全上下文（HTTPS 或 localhost）。如果通过 HTTP 访问，摄像头权限会被直接拒绝。AutoDL 的 6006 端口自带 HTTPS 支持，通过 Nginx 代理前端和后端，完美解决了这个问题。

7. API 接口一览

方法	路由	说明
GET	<code>/health</code>	健康检查

GET	<code>/api/v1/config</code>	当前配置
POST	<code>/api/v1/inference/emotion</code>	单帧推理
WS	<code>/ws/v1/inference/stream</code>	WebSocket 流式推理
GET	<code>/api/v1/dataset/info</code>	数据集元信息
GET	<code>/api/v1/dataset/class-distribution</code>	类别分布
GET	<code>/api/v1/stats/performance</code>	训练性能指标
GET	<code>/api/v1/stats/confusion-matrix</code>	混淆矩阵
GET	<code>/api/v1/stats/latency</code>	推理延迟

8. AutoDL 部署流程



复制代码

1. 克隆项目

```
cd /root/autodl-tmp
git clone https://github.com/linzzzuan/Art_ai_final.git emotion-project
cd emotion-project
```

2. 后端环境

```
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && pip install mediapipe
```

3. 修改 config.yaml → active_profile: server

4. 预计算几何特征

```
python scripts/precompute_geo_features.py --dataset-path datasets/real
```

5. 训练

```
rm -f checkpoints/best_model.pth
python train.py --dataset-path datasets/real --epochs 60 --batch-size 64 \
    --lr 0.0005 --task-name resnet18_v2 --num-workers 4
```

6. 前端构建

```
cd ../frontend && npm install && npm run build
```

7. Nginx 代理 + 启动后端

9. 总结：AI 编程的核心价值

我做了什么

我做了四件事：提问题、做决策、验收结果、把握方向。

提问题：我不会写 PyTorch 模型，但我知道 46% 的准确率太低了。我不会调 CSS，但我知道特征点看不清。我不会排查分布偏移，但我知道"微笑被识别为恐惧"这个现象不对。问题描述得越具体，AI 给出的解决方案就越精准。

做决策：AI 会给出多个方案供我选择——用 ResNet18 还是 ResNet50？冻结 4 层还是 6 层？用 Adam 还是 AdamW？最终拍板的是我。决策权在我手里，AI 提供的是选项和论据。

验收结果：每个阶段我都会验证——准确率是否提升？推理是否正常？特征点是否可见？不合格就继续迭代。AI 不会自动判断好坏，它需要我的反馈来调整方向。

把握方向：我从一开始就设定了"全栈闭环"的目标——不是只做一个模型，而是做一个完整的产品。这个方向是我定的，AI 沿着这个方向填充了每一层实现。

AI 做了什么

AI 做了三件事：设计架构、编写代码、调试问题。

设计架构：AI 设计了整个系统的架构——前端用 React + TypeScript + MediaPipe，后端用 FastAPI + PyTorch，模型用 ResNet18 + GeoEncoder + FusionClassifier。这不是简单的代码堆砌，而是一个前后端契约一致、数据流清晰的完整架构。

编写代码：从模型定义（ResNet18 backbone 替换 4 层 CNN）、训练管线（TrainingEngine）、推理服务（ModelService）、API 路由（9 个接口），到前端 Hooks（4 个）、页面组件（3 个）、可视化图表（3 个）、数据处理脚本（2 个）、部署脚本（2 个）——全部由 AI 编写。总代码量超过 3000 行，覆盖了数据、模型、训练、推理、前端、部署的完整链路。

调试问题：每一次准确率不达标、推理结果偏差、联调不一致，AI 都能给出根因分析。"微笑被误识别为恐惧"这个问题，AI 从类别权重、冻结层、几何特征分布三个维度定位了根因，并给出了 7 项系统性的修复方案。传统开发中这需要多次沟通、多次验证，AI 通过上下文理解一次性给出了解决方案。

AI 编程的本质

你提出问题和目标，**AI** 提供从架构到代码到调试的端到端解决方案。它不是工具，而是合作伙伴——理解你的业务需求、设计系统架构、编写跨层代码、诊断复杂问题。更重要的是：它让一个人，完成了原本需要 CV 算法工程师 + 后端开发 + 前端开发 + 运维工程师才能完成的工作。